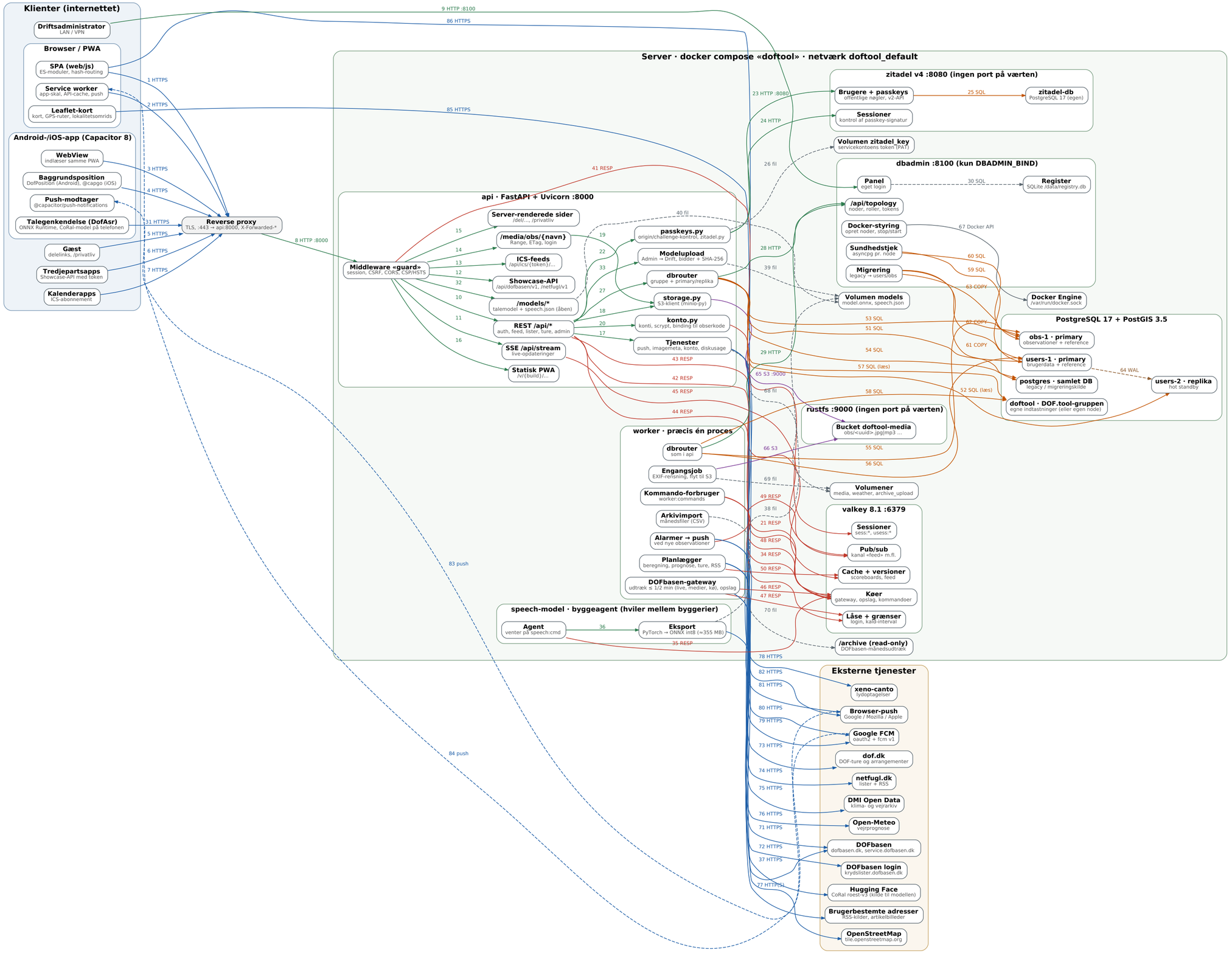
DOF.tool – kommunikationsveje

9. november 2026 · kode fbfe1aa

Alle forbindelser mellem klienter, containere, delelementer i containerne og eksterne tjenester. Tallet på hver pil henviser til tabellen «Kommunikationsveje», der beskriver type og indhold. Farven angiver protokollen.



Teknisk dokumentation af tech-stacken

DOF.tool er en portal for feltornitologer: live-observationer fra DOFbasen, egne feltmeldinger og ture, lister, kalendere, fænologi, læring og showcase-moduler for DOFbasen og Netfugl. Den kører som en samling Docker-containere bag en reverse proxy og leveres som PWA i browseren og som Capacitor-app på Android/iOS.

Kommunikationsveje

HTTPS HTTP Kald PostgreSQL Replikering RESP S3 Docker API Fil Push

#	Fra	Til	Type	Indhold
1	SPA (web/js) (Browser / PWA)	Reverse proxy	HTTPS	HTML/JS/CSS, REST-JSON, multipart-upload af fotos/lyd, cookie dt_session
2	Service worker (Browser / PWA)	Reverse proxy	HTTPS	App-skal (netværk først, cache som nødløsning), API-svar caches offline
3	WebView (Android-/iOS-app)	Reverse proxy	HTTPS	Samme trafik som browseren – appen indlæser PWA'en fra serveren
4	Baggrundsposition (Android-/iOS-app)	Reverse proxy	HTTPS	POST /api/location/native: position + positionstoken, også når appen er lukket; GPS kun under ruteoptagelse, Nær mig via Wi-Fi/master
5	Gæst	Reverse proxy	HTTPS	GET delelinks (/del/traad/…) og /privatliv uden login
6	Tredjepartsapps	Reverse proxy	HTTPS	GET JSON fra showcase-API'erne med Authorization: Bearer <token>; åben CORS, kun læsning
7	Kalenderapps	Reverse proxy	HTTPS	GET text/calendar (ICS) med personligt token i adressen
8	Reverse proxy	Middleware «guard» (api)	HTTP	Al klienttrafik; proxyen sætter X-Forwarded-For/-Proto (stoles kun på fra FORWARDED_ALLOW_IPS)
9	Driftsadministrator	Panel (dbadmin)	HTTP	db-admin-panelet og dets JSON-API; eget kodeord, cookie med SameSite=Strict
10	Middleware «guard» (api)	REST /api/* (api)	Kald	Godkendte forespørgsler til REST-routerne
11	Middleware «guard» (api)	SSE /api/stream (api)	Kald	Server-sent events til indloggede (loft over åbne strømme; lukkes efter 30 s i baggrunden)
12	Middleware «guard» (api)	Showcase-API (api)	Kald	Showcase-API (token kontrolleres i routeren)
13	Middleware «guard» (api)	ICS-feeds (api)	Kald	ICS-feeds
14	Middleware «guard» (api)	/media/obs/{navn} (api)	Kald	Filer fra observationer – kun med gyldig session
15	Middleware «guard» (api)	Server-renderede sider (api)	Kald	Delelinks og privatlivspolitik

#	Fra	Til	Type	Indhold
16	Middleware «guard» (api)	Statisk PWA (api)	Kald	Statiske app-filer med build-id i adressen
17	REST /api/* (api)	Tjenester (api)	Kald	Alarmer/push, metadata-rensning af uploads, udtræk og sletning af konto, lagerforbrug
18	REST /api/* (api)	storage.py (api)	Kald	Gem/slet filer ved upload til feltmeldinger og ture
19	/media/obs/{navn} (api)	storage.py (api)	Kald	Hent fil (evt. et byte-interval) til afspilning/visning
20	REST /api/* (api)	konto.py (api)	Kald	Login med e-mail/obserkode + adgangskode, opret konto, glemt/skift adgangskode (/api/auth/login, /api/auth/konto/*, /api/me/password)
21	konto.py (api)	Køer (valkey 8.1)	RESP	Bind kontoen: ét DOFbasen-login som gateway-job; engangsbevis som hash (konto:bind:…, 20 min)
22	REST /api/* (api)	passkeys.py (api)	Kald	Login med passkey og brugerens egne passkeys (/api/auth/passkey/*, /api/me/passkeys)
23	passkeys.py (api)	Brugere + passkeys (zitadel v4)	HTTP	Opret bruger (obserkode), start/bekræft/slet passkey; Bearer-token fra zitadel_key
24	passkeys.py (api)	Sessioner (zitadel v4)	HTTP	Session med passkey-udfordring → telefonens signatur kontrolleres; sessionen slettes bagefter
25	Brugere + passkeys (zitadel v4)	zitadel-db (zitadel v4)	PostgreSQL	Zitadels egne data (brugere, offentlige nøgler, hændelser), krypteret med ZITADEL_MASTERKEY
26	passkeys.py (api)	Volumen zitadel_key	Fil	Læser servicekontoens token (read-only), som Zitadel skrev ved første start
27	REST /api/* (api)	dbrouter (api)	Kald	SQLAlchemy async-sessioner; hver sætning routes til sin databasegruppe
28	dbrouter (api)	/api/topology (dbadmin)	HTTP	GET /api/topology hvert 15. s med Bearer servicetoken → JSON: noder, roller, adresser, adgangskoder
29	dbrouter (worker)	/api/topology (dbadmin)	HTTP	Samme som api
30	Panel (dbadmin)	Register (dbadmin)	Fil	Registret over noder, indstillinger og sessioner (lokal SQLite-fil)
31	Talegenkendelse (DofAsr) (Android-/iOS-app)	Reverse proxy	HTTPS	GET /models/speech.json og model.onnx (≈355 MB) én gang; derefter genkendes talen på telefonen – lyden forlader den ikke
32	Middleware «guard» (api)	/models/* (api)	Kald	Talemodellen leveres uden login (åben model) med lang cache
33	REST /api/* (api)	Modelupload (api)	Kald	Upload af en færdigbygget model i bidder (kræver Drift-rettighed)
34	Kommando-forbruger (worker)	Køer (valkey 8.1)	RESP	RPUSH speech:cmd: start modelbyggeri (Kør nu i Admin → Drift eller ugentlig plan)

#	Fra	Til	Type	Indhold
35	Agent (speech-model)	Køer (valkey 8.1)	RESP	BLPOP speech:cmd; byggestatus og log skrives til worker:jobs, så de vises i Drift
36	Agent (speech-model)	Eksport (speech-model)	Kald	Kører eksporten; afbryder tydeligt, hvis hukommelsen er for lille (≈6,5 GB fri)
37	Eksport (speech-model)	Hugging Face	HTTPS	GET modelvægte, ordliste og konfiguration ved en fastlåst revision (kun ved byggeri)
38	Eksport (speech-model)	Volumen models	Fil	Skriver model.onnx og manifestet speech.json
39	Modelupload (api)	Volumen models	Fil	Lægger den uploadede model på plads efter kontrol af indhold og SHA-256
40	/models/* (api)	Volumen models	Fil	Læser modelfilerne fra volumen models (read-only)
41	Middleware «guard» (api)	Sessioner (valkey 8.1)	RESP	Slår sessionen op under hashen af cookien (sess:h:<sha256> → bruger-id)
42	REST /api/* (api)	Køer (valkey 8.1)	RESP	ZADD gateway-job (fx kontobinding – DOFbasen-adgangskoden ligger højst 3 min), RPUSH worker-kommandoer
43	REST /api/* (api)	Cache + versioner (valkey 8.1)	RESP	GET/SET JSON-cache, versionsnumre, admin-målinger
44	REST /api/* (api)	Låse + grænser (valkey 8.1)	RESP	Tællere for login-forsøg og andre grænser
45	SSE /api/stream (api)	Pub/sub (valkey 8.1)	RESP	SUBSCRIBE på live-kanaler; beskeder sendes videre som SSE
46	DOFbasen-gateway (worker)	Køer (valkey 8.1)	RESP	ZPOPMIN næste job; LPUSH svar til den api-proces, der venter
47	DOFbasen-gateway (worker)	Låse + grænser (valkey 8.1)	RESP	Globalt tidsstempel + lås: højst ét udtræk pr. minut, også ved flere workers
48	Kommando-forbruger (worker)	Køer (valkey 8.1)	RESP	BLPOP worker:commands (fx genberegning, arkivimport)
49	Alarmer → push (worker)	Pub/sub (valkey 8.1)	RESP	PUBLISH nye fund og ændringer til live-feedet
50	Planlægger (worker)	Cache + versioner (valkey 8.1)	RESP	Bump cache-versioner, jobstatus til Admin → Drift
51	dbrouter (api)	users-1 · primary (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	Læs/skriv brugerdata (konti med e-mail og scrypt-hash, lister, feltmeldinger, kommentarer …) via asyncpg
52	dbrouter (api)	users-2 · replika (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	Læsninger af brugerdata, når «læs fra replikaer» er slået til
53	dbrouter (api)	obs-1 · primary (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	Læs observationer, aggregater og referencedata
54	dbrouter (api)	postgres · samlet DB (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	Al data i legacy-tilstand (før opdelingen er slået til)

#	Fra	Til	Type	Indhold
55	dbrouter (worker)	obs-1 · primary (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	INSERT/COPY af udtræk og arkiv, natlige aggregater
56	dbrouter (worker)	users-1 · primary (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	Alarmregler, notifikationer, oprydning
57	dbrouter (api)	doftool · DOF.tool-gruppen (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	Søg, Observationer, lister og statistik læser DOF.tool-gruppen sammen med DOFbasens data (dataset.py)
58	dbrouter (worker)	doftool · DOF.tool-gruppen (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	Afstemning: ture og fund fra brugergruppen skrives som observationer/ture (ved ændring og hvert 10. min)
59	Sundhedstjek (dbadmin)	users-1 · primary (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	SELECT 1 + svartid → nodens status i panelet
60	Sundhedstjek (dbadmin)	obs-1 · primary (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	Samme sundhedstjek
61	Migrering (dbadmin)	postgres · samlet DB (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	COPY ud af den samlede database (kilde)
62	Migrering (dbadmin)	users-1 · primary (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	COPY ind i brugergruppen; derefter indekser
63	Migrering (dbadmin)	obs-1 · primary (PostgreSQL 17 + PostGIS 3.5)	PostgreSQL	COPY ind i observationsgruppen i parallelle intervaller
64	users-1 · primary (PostgreSQL 17 + PostGIS 3.5)	users-2 · replika (PostgreSQL 17 + PostGIS 3.5)	Replikering	Første gang pg_basebackup, derefter streaming-replikering af WAL
65	storage.py (api)	Bucket doftool-media (rustfs)	S3	PUT/GET (med Range)/HEAD/DELETE objekter, signeret med SigV4
66	Engangsjob (worker)	Bucket doftool-media (rustfs)	S3	Engang: flyt eksisterende filer fra media/obs til bucket'en
67	Docker-styring (dbadmin)	Docker Engine	Docker API	Opret databasecontainere, stop/start api og worker under migrering
68	Tjenester (api)	Volumener	Fil	Netfugl-fotos, læringsfiler og xeno-canto-cache på volumen media
69	Engangsjob (worker)	Volumener	Fil	Engang: fjern EXIF/GPS fra billeder uploadet før rensningen
70	Arkivimport (worker)	/archive (read-only)	Fil	Læs dofbasen_ÅÅÅÅ_MM.csv (read-only bind-mount)
71	DOFbasen-gateway (worker)	DOFbasen	HTTPS	GET observationer, ture, referencedata, medie-scanning – højst ét udtræk pr. 2 min (delt)
72	DOFbasen-gateway (worker)	DOFbasen login	HTTPS	POST JSON obserkode + adgangskode, kun når en konto oprettes eller får ny adgangskode (adgangskoden gemmes ikke)

#	Fra	Til	Type	Indhold
73	Planlægger (worker)	dof.dk	HTTPS	GET HTML med DOF-ture og arrangementer (hver 6. time)
74	Planlægger (worker)	netfugl.dk	HTTPS	GET RSS/HTML (hver time) og Netfugl-lister
75	Planlægger (worker)	DMI Open Data	HTTPS	GET klimadata og vejrarkiv (JSON/månedsfiler)
76	Planlægger (worker)	Open-Meteo	HTTPS	GET vejrprognose (JSON, hver time)
77	Tjenester (api)	Brugerbestemte adresser	HTTPS	RSS og billeder i importerede artikler; SSRF-værn (egress.py) afviser interne adresser, også ved omdirigering
78	Tjenester (api)	xeno-canto	HTTPS	GET lydfiler til Mobgang; caches på volumen media/xc
79	Alarmer → push (worker)	Google FCM	HTTPS	OAuth2-JWT til oauth2.googleapis.com + push via FCM v1 (JSON)
80	Tjenester (api)	Google FCM	HTTPS	Push ved feltmeldinger, kommentarer og statusmeldinger
81	Alarmer → push (worker)	Browser-push	HTTPS	Web Push: VAPID-signeret, krypteret payload til browserens push-tjeneste
82	Tjenester (api)	Browser-push	HTTPS	Web Push fra api (samme som worker)
83	Google FCM	Push-modtager (Android-/iOS-app)	Push	Notifikation leveres til appen
84	Browser-push	Service worker (Browser / PWA)	Push	push-event i service workeren → notifikation
85	Leaflet-kort (Browser / PWA)	OpenStreetMap	HTTPS	GET kortfliser (PNG) direkte fra browseren – brugerens IP sendes til OSMF
86	SPA (web/js) (Browser / PWA)	DOFbasen	HTTPS	Billeder og lyd fra DOFbasen vises direkte i browseren

Containere

Container	Image / bygget af	Rolle	Porte	Volumener
api	`server/Dockerfile` (python:3.12-slim)	FastAPI-app: REST, SSE, showcase-API'er, server-renderede sider, filer fra observationer og den statiske PWA.	`PORT` → 8000 (til proxyen)	media, weather, archive_upload, `/archive` (ro), `web/` (ro)
worker	samme image som api	Præcis én proces: DOFbasen-gateway, planlagte job, arkivimport, alarmer og push, engangsmigreringer.	ingen	som api
valkey	`valkey/valkey:8.1`	Sessioner, køer, cache, låse, pub/sub. AOF-persistens, 512 MB med `volatile-lru`.	kun internt	valkeydata

Container	Image / bygget af	Rolle	Porte	Volumener
postgres	`postgis/postgis:17-3.5`	Den samlede database: al data i legacy-tilstand, kilde og sikkerhedskopi efter opdelingen.	kun internt	pgdata
doftool-db-*	`doftool-dbnode` (PostGIS + replikering)	Databasenoder i grupperne users, observations og doftool; primary eller replika. Oprettes af db-admin eller med dens script.	ingen / `127.0.0.1` på samme vært	doftool_db_<navn>
rustfs	`rustfs/rustfs:1.0.3`	Objektlager (S3) til fotos og lyd i observationer, bucket `doftool-media`.	ingen på værten	rustfs
dbadmin	`dbadmin/Dockerfile`	Register over databasenoder, topologi til api/worker, sundhedstjek, migrering, oprettelse af noder via Docker.	`DBADMIN_BIND`:8100 (standard 127.0.0.1)	dbadmin, `docker.sock`
zitadel + **zitadel-db**	`ghcr.io/zitadel/zitadel:v4.20.2` (AGPL-3.0, uændret), `postgres:17-alpine`	Gemmer brugernes passkeys (offentlige nøgler) og kontrollerer signaturerne. Kun api'et kalder dens v2-API; Zitadels egne login-sider og konsol bruges ikke. ≈100 MB hukommelse hver.	ingen	zitadel_db, zitadel_key
speech-model	`tools/speech-model/Dockerfile` (PyTorch, Hugging Face-værktøj)	Byggeagent: bygger talemodellen til appens tale-indtastning (CoRal → ONNX int8) på kommando fra Admin → Drift. Hviler mellem byggerier; bruger under et byggeri op til ca. 6 GB hukommelse.	ingen	models, speech_work

Alle containere logger til Docker med rotation (10 MB × 5 filer). Reverse proxyen (TLS) ligger uden for compose-projektet.

Teknologier

Lag	Teknologi	Bemærkning
Sprog og runtime	Python 3.12	Asynkron hele vejen (asyncio).
Web	FastAPI 0.159, Starlette 1.9, Uvicorn 0.54	Uvicorn stoler kun på proxy-headers fra `FORWARDED_ALLOW_IPS` (standard: private net).
Database-adgang	SQLAlchemy 2.1 (async) + asyncpg 0.32	Egen `RoutingSession` sender hver sætning til sin databasegruppe.

Lag	Teknologi	Bemærkning
Database	PostgreSQL 17 + PostGIS 3.5	Al rumlig logik (områder, radius, kommuner) i PostGIS.
Cache, køer, pub/sub	Valkey 8.1 (redis-py 8.1)	Open source-afløseren for Redis.
Objektlager	RustFS 1.0.3 (Apache-2.0), S3-klienten minio-py 7.2.20	Afløste MinIO (se note nedenfor).
Validering og config	Pydantic 2.14, pydantic-settings	Al konfiguration fra miljøvariabler/ `.env`.
Udgående HTTP	httpx 0.28	DOFbasen, DMI, Open-Meteo, xeno-canto, Netfugl, dof.dk.
Push	pywebpush 2.5 + py-vapid 1.9 (Web Push), FCM HTTP v1	VAPID-nøglen ligger i databasen (`settings`), FCM-servicekontoen i `.env`.
Øvrigt	numpy, orjson, cryptography	Fænologi-beregninger, hurtig JSON, kryptering.
Frontend	Vanilla JavaScript (ES-moduler), Leaflet 1.9.4, lamejs	Ingen byggetrin; filerne serveres med build-id i adressen. Service worker til offline og push.
Mobilapp	Capacitor 8 (Android/iOS)	WebView med samme PWA; baggrundsposition og push-notifikationer. På Android bruges appens eget positionsplugin `DofPosition` (fused-positionstjenesten, ingen wakelock), på iOS `@capgo/background-geolocation`. Android bygges lokalt (`app/build.sh`), iOS som kildepakke til Xcode eller via GitHub Actions.
Tale-indtastning	CoRal `roest-v3-wav2vec2-315m` (ONNX int8), ONNX Runtime (MIT)	Kører på telefonen i pluginnet `DofAsr`; ingen Google-genkender. Output afkodes i `web/js/voice-ctc.js` mod artslisten, tal og adfærdskoder.
Drift	Docker Compose	db-admin styrer databasenoder via Docker API'et.

Data og databaser

Tabellerne er delt i grupper (`server/portal/dbgroups.py`):

- **users** – alt der handler om brugere: konti, lister, områder, alarmregler, feltmeldinger, kommentarer, kalendere, Netfugl-indhold, support.
- **observations** – observationer, ture og observatører fra DOFbasen samt alle aggregater (artsdage, fænologi, regioner, vejr).
- **doftool** – observationer og ture indtastet i DOF.tool, i samme skema som DOFbasens, men i sin egen database (uden egen node en separat database på observationsgruppens server). Den bygges af workeren ud fra turene og fundene i brugergruppen (ved hver ændring og hvert 10. minut) og kan altid genskabes. Står samme fugl også i DOFbasen, tæller kun DOFbasens række. Søg, Observationer, lister og statistik kører samme forespørgsel i begge grupper og fletter svarene (`services/dataset.py`); hemmelige DOF.tool-fund er udeladt for andre end ejeren.
- **reference** – arter, afdelinger, kommuner, lokaliteter m.m. findes i begge grupper, så joins mod dem er lokale.

En SQL-sætning må aldrig blande bruger- og observationstabeller; routeren afviser den. Data hentes i stedet i to trin (fx et tegnet områdes geometri slås op i brugergruppen og sendes som parameter til observationsforespørgslen).

Hver gruppe har én primary og nul eller flere replikaer. Replikaer tager en `pg_basebackup` af primary og streamer derefter WAL. En replika er enten til at aflaste læsninger eller kun til backup. Topologien ligger i db-admins register; api og worker henter den hvert 15. sekund og skifter forbindelser uden genstart. I legacy-tilstand kører alt på den samlede database (`postgres`), som også er kilde, når db-admin migrerer.

Objektlager

Fotos og lyd i feltmeldinger og ture gemmes i RustFS under `obs/<uuid>.<endelse>`. Adressen er `/media/obs/<navn>`, og api'et leverer filen efter login med Range-understøttelse (lydafspilning), ETag og privat cache. Bucket'en er aldrig offentlig. Uden `S3_ENDPOINT` ligger filerne på disken som før. Netfugl-fotos og læringsfiler ligger stadig på volumenet `media`.

> **Vedligehold:** RustFS afløste MinIO, som ikke længere udgiver images og blev arkiveret i 2026. Filerne flyttes over én gang med `tools/minio-to-rustfs.sh`. Appen taler almindelig S3, så en anden tjeneste (Garage, SeaweedFS, Backblaze B2 …) kan overtage ved at ændre `S3_ENDPOINT`, `S3_ACCESS_KEY`, `S3_SECRET_KEY` og `S3_BUCKET`.

Konti, login og passkeys

Man logger ind med en **DOF.tool-konto**: e-mail eller obserkode og egen adgangskode (scrypt i `users.pw_hash`, mindst 10 tegn) – eller en passkey. Kontoen bindes til obserkoden ved oprettelsen (`services/konto.py`):

1. `POST /api/auth/konto/bekraeft`: obserkode + DOFbasen-adgangskode kontrolleres én gang hos DOFbasen gennem gatewayens opslagsbane (adgangskoden ligger højst 3 min i køen og gemmes ikke – heller ikke som hash). Svaret er et engangsbevis, der kun står som hash i Valkey (`konto:bind:<sha256>`, 20 min).
2. `POST /api/auth/konto/opret`: navn, e-mail og adgangskode. Felterne kontrolleres, før beviset bruges (GETDEL). Findes obserkoden i forvejen, beholdes brugerrækken med alle data. Alle brugerens tidligere sessioner og positionstokens tilbagekaldes, og der startes en ny session.
3. Tilbud om passkey på enheden.

Glemt adgangskode er samme vej: den, der kan logge ind på DOFbasen med obserkoden, kan vælge en ny adgangskode. Portalen sender ingen e-mails. Brugere fra før kontiene (kun obserkode, ingen `pw_hash`) får 409 ved login og sendes til oprettelsen; ved overgangen blev alle sessioner, positionstokens og passkey-udfordringer slettet én gang (markøren `accounts_logout_done` i `settings` tages atomisk). Profil → Konto og login skifter adgangskode (`/api/me/password` – logger andre enheder ud) og e-mail (`/api/me/email`).

En **passkey** (Face ID, fingeraftryk, skærmlås) kræver en konto. Alt går gennem api'ets egne endpoints (`/api/auth/passkey/*`, `/api/me/passkeys`); der åbnes ingen fremmede vinduer, og browseren og appen ser aldrig Zitadel.

- **Zitadel** gemmer de offentlige nøgler og kontrollerer signatur, tæller og brugerbekræftelse. Api'et kalder den med servicekontoens token (`services/zitadel.py`). Brugeren oprettes i Zitadel ved første passkey med obserkoden som brugernavn; id'et står i `users.zitadel_id`.
- **Api'ets egne kontroller** (`services/passkeys.py`): Zitadel kontrollerer ikke sidens oprindelse, så api'et kontrollerer selv `clientDataJSON` (type, origin = `PUBLIC_URL`, challenge), før svaret sendes videre. Udfordringer ligger i Valkey i 3–5 minutter og bruges én gang. En passkey kan kun tilføjes inden for 15 minutter efter et login.

- **Appen:** Android slår WebAuthn til i WebView'en (`androidx.webkit`), iOS har Associated Domains (`webcredentials:`). Serveren leverer `/.well-known/assetlinks.json` og `/.well-known/apple-app-site-association` ud fra `ANDROID_CERT_SHA256` og `IOS_TEAM_ID`. Passkeys er bundet til domænet – skifter portalen domæne, skal brugerne oprette nye.
- Sletning af kontoen sletter også brugeren og nøglerne i Zitadel.

Tale-indtastning og talemodel

Under en tur kan observationer indtales i appen. Talen genkendes **på telefonen**: pluginnet `DofAsr` optager 16 kHz mono og kører en dansk wav2vec2-model (CoRal `roest-v3`, ONNX int8) med ONNX Runtime. Lyden sendes aldrig til serveren eller til tredjepart. Modellens output afkodes mod artslisten, tallene og adfærdslisten, så kun ord derfra kan komme ud.

Modellen (≈355 MB) ligger ikke i appen. Den bygges af containeren `speech-model` på kommando fra Admin → Drift (workeren lægger kommandoen i Valkey-listen `speech:cmd`; også ugentligt, men en model bygget af samme revision springes over). Agenten henter CoRal-modellen fra Hugging Face ved en fastlåst revision, eksporterer den og lægger den i volumen `models` sammen med manifestet `speech.json`. Api'et leverer filerne under `/models/`, og appen henter dem én gang og tilbyder opdatering, når manifestet ændres. Har serveren for lidt hukommelse, bygges modellen på en anden maskine (`tools/speech-model/pack.sh`) og uploades under Admin → Drift i bidder; arkivet kontrolleres (kun forventede filer, SHA-256), før det tages i brug.

Ture og GPS-ruter

En tur sendes og gemmes i ét kald og kan lægges i en lokal kø på enheden (IndexedDB), når der ikke er net. Nye ture oprettes altid på brugerens egen obserkode. Ruten optages i appen af den native positionstjeneste (også med slukket skærm; GPS hvert 5. s/5. m, i baggrunden sendt samlet pr. minut) og i browseren af `navigator.geolocation`. Når turen afsluttes, stoppes optagelsen, og ruten beskæres til turens start- og sluttidspunkt. Efter afslutningen tilbyder appen de 30 mest almindelige arter på lokaliteten (`GET /api/trips/common`, bygget på lokalitetens cachede artsliste), som endnu ikke står på turen, til hurtig indtastning af antal.

Kort: «Se på kort» viser turens GPS-rute på et Leaflet-kort under turens data. Når en lokalitet vælges på kortet, tegnes lokaliteternes omrids (`GET /api/locations/shapes` for kortudsnittet, fra zoom 10), og punktet vælger den lokalitet, det ligger i – den inderste først, ellers den med den nærmeste grænse (`/api/reports/near`). Kortfliserne hentes direkte fra OpenStreetMap. Ruten og positioner vises ikke i «se som bruger».

Baggrundsjob

Job	Hvornår	Hvad
Live-hentning	løbende (efter plan i Admin → Data)	Dagens observationer fra DOFbasen via gatewayen; nye fund udløser alarmer, push og live-feed.
DOFbasen-gateway	altid	Højst ét udtræk hvert andet minut (globalt tidsstempel + lås i Valkey). Live-hentning, medie-scanning og kø deler pladserne. Login/kontobinding og billedopslag for én observation har en separat kø uden interval.
Medie-scanning	følger live-planen, højst hvert 10. min	`observationer/index.php`: billeder, lyd og DKU-status for dagens observationer – som regel de sjældne, hver 3. gang alle fugle. Et job i gatewayens udtræksbane.
Vejrprognose	hver time (efter :15)	Open-Meteo.

Job	Hvornår	Hvad
Natlig beregning	kl. 02:30	Dagsaggregater, lokale tal, fænologi, sæsoner, arter pr. år, oprydning (notifikationer 90 d, aktivitet 180 d, positioner 30 d uden opdatering).
Referencedata	kl. 03:00 + hver time til databasegrupperne	Arter, lokaliteter m.m. fra DOFbasen og synkronisering mellem grupperne.
Hul-udfyldning	hver 6. time	Henter måneder/dage, der mangler.
DOF-ture	hver 6. time	dof.dk.
Klima	mandag kl. 05	DMI-klimadata.
Netfugl	krydslister hver 6. time, RSS hver time	Kun når modulet er slået til.
Talemodel	ugentligt + «Kør nu»	Starter modelbyggeriet i `speech-model`; springes over, hvis modellen allerede er bygget af samme revision.
Engangsjob	ved første start	Fjern metadata fra eksisterende billeder; flyt filer fra disken til objektlageret.

Sikkerhed

- **Login** med DOF.tool-kontoens adgangskode (scrypt, eget salt; ukendt bruger koster samme tid). DOFbasen-adgangskoden bruges kun til at binde kontoen og gemmes ikke. Se «Konti, login og passkeys». Højst 5 forsøg i minuttet pr. konto og pr. IP og 30 pr. konto i timen; kontoen indgår kun som hash i tællernes nøgler.
- **Sessioner** i Valkey (`sess:h:<sha256 af cookien>` – kun hashen gemmes), cookie `HttpOnly`, `SameSite=Lax`, altid `Secure` når `PUBLIC_URL` er https. Alle en brugers sessioner kan lukkes på én gang. Positionstokens gemmes også kun som hash.
- **CSRF**: ændrende kald kræver `X-Requested-With` eller `Sec-Fetch-Site: same-origin`.
- **Headers**: streng CSP (scripts kun egne + hash), HSTS over https, `X-Frame-Options: DENY`, `nosniff`, Referrer- og Permissions-Policy. Uploads leveres med `sandbox`-CSP.
- **Klientens IP** tages kun fra X-Forwarded-For, når kaldet kommer fra en betroet proxy.
- **Uploads**: størrelsesgrænse, tilladte typer, tilfældigt filnavn; GPS og andre metadata fjernes, før filen gemmes.
- **SQL**: kun parametre; sortering og visning fra whitelists. Regnearkseksporter neutraliserer formler.
- **Udgående kald** til brugerbestemte adresser (RSS, artikelbilleder) går gennem SSRF-værnet (`services/egress.py`), som afviser interne adresser, også efter omdirigering.
- **«Se som bruger»** er rent læsende og viser hverken positioner, ruter, ICS-link eller personlige notifikationer. Ændringer af rettigheder skrives i en audit-log.
- **Afhængigheder** er låst (`server/constraints.txt`); der er loft over åbne SSE-strømme pr. bruger.
- **Showcase-API'er**: token gemt som hash, kun læsning, egne grænser pr. minut.
- **Netværk**: db-admin, databasenoder, RustFS og Zitadel har ingen offentlige porte; db-admin har eget kodeord og begrænsning af forsøg. db-admin har adgang til Docker og skal aldrig udstilles mod internettet.

Persondata

Privatlivspolitikken ligger på `/privatliv`. Brugeren kan selv hente alle sine data som JSON (`GET /api/me/export`) og slette sin konto, hvilket også sletter uploadede filer, aktivitetslog og alle sessioner. Der er ingen analyse- eller sporingsværktøjer og kun én (nødvendig) cookie. Data sendes til tredjeparter som

vist i diagrammet: DOFbasen (login, udtræk), OpenStreetMap (kortfliser, brugerens IP), Google FCM og browserens push-tjeneste (notifikationer). Hugging Face kontaktes kun af byggeagenten ved modelbyggeri, aldrig med brugerdata, og indtalt lyd forlader ikke telefonen.

Drift

Vigtige miljøvariabler (`.env`)

Variabel	Betydning
`PUBLIC_URL`, `PORT`	Portalens adresse og api'ets port mod proxyen.
`POSTGRES_*`	Den samlede database.
`DBADMIN_URL`, `DBADMIN_SERVICE_TOKEN`, `DBADMIN_PASSWORD`, `DBADMIN_BIND`	Registret, token mellem api/worker og db-admin, panelets kodeord og adresse.
`RUSTFS_ACCESS_KEY`, `RUSTFS_SECRET_KEY`, `S3_*`	Objektlageret.
`FORWARDED_ALLOW_IPS`	Proxyer, hvis X-Forwarded-For stoles på.
`PRIVACY_CONTROLLER`, `PRIVACY_EMAIL`, `PRIVACY_BACKUP_DAYS`	Oplysninger i privatlivspolitikken.
`DOF_ARCHIVE_DIR`, `DOF_MIN_INTERVAL`, `DOF_ENABLED`	DOFbasen-arkivet og gatewayens interval (mindst 120 s).
`FCM_SERVICE_ACCOUNT_B64`, `VAPID_SUBJECT`	Push.
`ZITADEL_MASTERKEY`, `ZITADEL_DB_PASSWORD`	Passkeys: Zitadels hovednøgle (32 tegn – kopi uden for serveren) og databasens kodeord.
`ANDROID_CERT_SHA256`, `IOS_TEAM_ID`, `PASSKEY_DOF_MAX_DAYS`	Passkeys i appen (domænets tilknytning) og dage mellem DOFbasen-bekræftelser.

Android-appen bygges med `app/build.sh` (signeret APK). iOS kan kun bygges på macOS; kildepakken `DOF-tool-ios.zip` åbnes i Xcode, eller GitHub Actions-workflowen `ios.yml` bygger den. Appen indlæser portalen fra serveren, så ændringer i web-delen kommer med serveren, ikke med en ny app.

Overvågning

Admin → Drift viser gateway, kø, job og **lagerplads** (disk, hver databasenode med største tabeller, objektlager, filer på disken og Valkey). db-admin viser hver nodes status og svartid. Sundhedstjek: `GET /api/health`.

Backup

Replikaer beskytter mod nedbrud af en node, ikke mod fejl i data eller tab af hele serveren. Brug derudover et natligt `pg_dump` af hver gruppe og en kopi af bucket'en, krypteret og gemt uden for serveren. Zitadel: `pg_dump` af `zitadel-db` plus `ZITADEL_MASTERKEY` (uden nøglen kan dumpet ikke bruges). Mistes Zitadel, logger brugerne ind med DOFbasen igen og opretter nye passkeys.